ADITYA UNIVERSITY

Advanced Data Structures & Algorithm Analysis (241CS010)

UNIT-II

Prepared by
Anil Kumar Prathipati,
Assistant Professor,
Website/blog: anilkumarprathipati.wordpress.com
ADITYA University

UNIT-II SYLLABUS

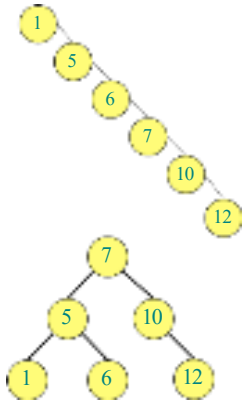
AVL Trees Creation, Insertion, Deletion operations and Applications.
B-Trees Creation, Insertion, Deletion operations and Applications.
Priority Queues (Heaps): Introduction, Binary Heaps-Model and Simple Implementation, Basic Heap Operations, Other Heap Operations, Applications of Priority Queues.

Outcome: Illustrate the concepts of search trees and operations of Priority Queues.

Text Book: Fundamentals of Data Structures in C++, Horowitz, Ellis; Sahni, Sartaj; Mehta, Dinesh, 2ndEdition Universities Press.

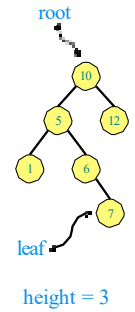
AVL Trees Need?

- Review: Binary Search Trees
- Importance of being balanced
- Balanced BSTs
  - AVL trees
    - definition
    - rotations, insert



Binary Search Trees (BSTs)

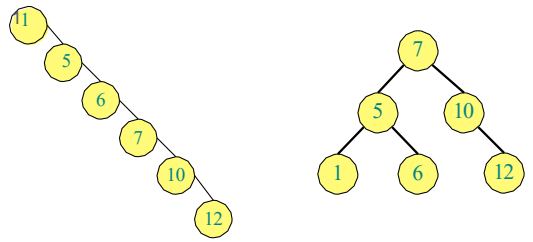
- Each node x has:
  - key[x]
  - Pointers: left[x], right[x], p[x]
- Property: for any node x:
  - For all nodes y in the left subtree of x: key[y] ≤ key[x]
  - For all nodes y in the right subtree of x: key[y] ≥ key[x]



height = 3

The importance of being balanced

for n nodes:



h = Θ(log n) h = Θ(n)

AVL Trees: Definition

[Adelson-Velskii and Landis]

An AVL tree is one that has one of the following characteristics at each of its nodes.

- If a node's longest path in its left subtree is longer than its longest path in its right subtree, the node is said to be "left heavy."
- If the longest path in a node's right subtree is one more long than the longest path in its left subtree, the node is said to be "right heavy."
- If the longest paths in the right and left subtrees are equal, a node is said to be balanced.
- **The AVL tree is a height-balanced tree where each node's left and right subtree height differences are either -1, 0 or 1.**

Balance factor properties

Balance Factor=Height of Left Subtree - Height of Right Subtree.

- If a subtree has a balance factor greater than zero, it is said to be left-heavy.
- If a subtree has a balance factor of 0, it is said to be right-heavy.
- If the balance factor is zero, the subtree is perfectly balanced.

Program Logic Support functions

```
struct Node {
    int key;
    struct Node *left, *right;
    int height;
};
int max(int a, int b) {
    return (a > b) ? a : b;
}
int height(struct Node *n) {
    return (n == NULL) ? 0 : n->height;
}
struct Node* newNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}
struct Node* minValueNode(struct Node* node) {
    struct Node* current = node;
    while (current->left != NULL)
        current = current->left;
    return current;
}
```

AVL Tree Operations

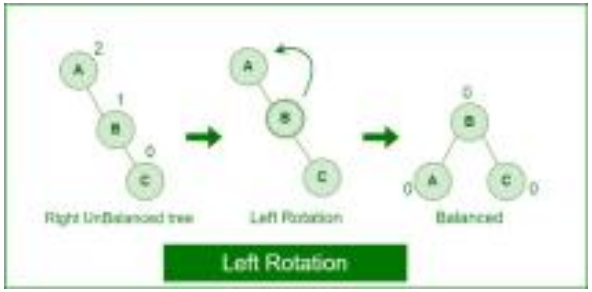
Three types of operations that are performed:

1. A New Node is Inserted.
2. Removal of a Node.
   - removing a node from the right subtree.
   - removing of a Node from the Left Subtree.
3. Looking for a Node in an AVL Tree.

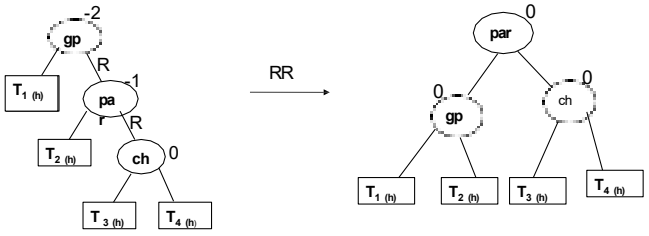
RR Rotation (Left Rotation)

There are basically four types of rotations which are as follows:

1. R R rotation : Inserted node is in the right subtree of right subtree of A. (Example: A, B, C.)



Contd …



RR Rotation tilt the tree towards left

Program Logic – Left Rotation

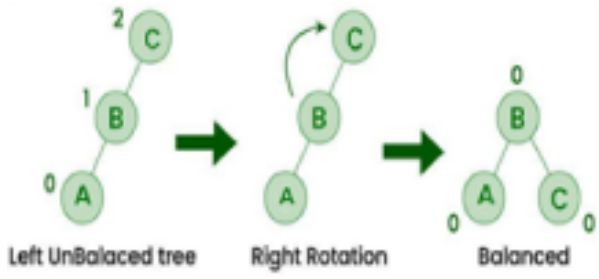
```
struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(height(x->left),  height(x->right)) + 1;
    y->height = max(height(y->left),  height(y->right)) + 1;
    return y;
}

int getBalance(struct Node* n) {
    return (n == NULL) ? 0 : height(n->left) - height(n->right);
}
```

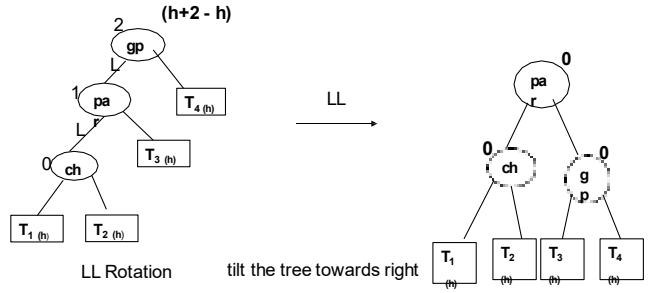
LL Rotation (Right Rotation)

There are basically four types of rotations which are as follows:

1. L L rotation: Inserted node is in the left subtree of left subtree of A. (Example: C, B, A.)



Contd …



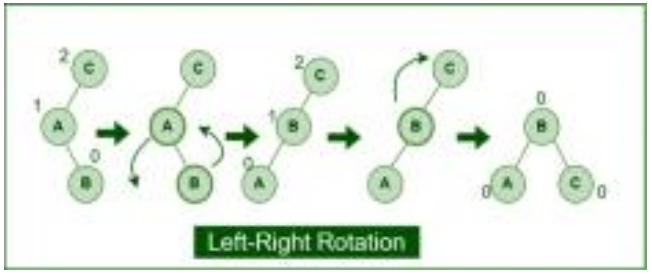
Program Logic – Right Rotation

```
struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(height(y->left),  height(y->right)) + 1;
    x->height = max(height(x->left),  height(x->right)) + 1;
    return x;
}
```

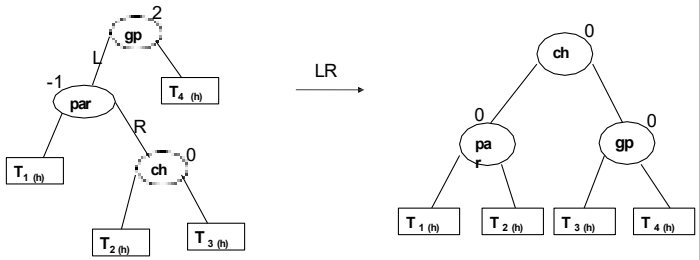

LR Rotation

There are basically four types of rotations which are as follows:

3. L R rotation : Inserted node is in the right subtree of left subtree of A. (Example: C, A, B.)



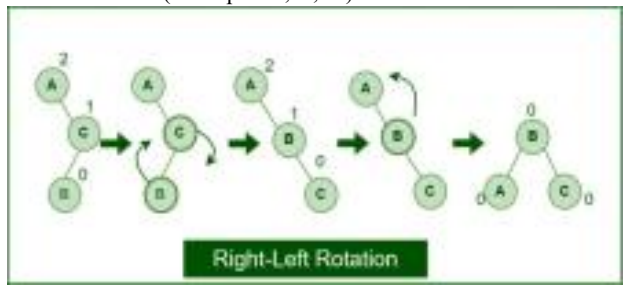
Contd ...



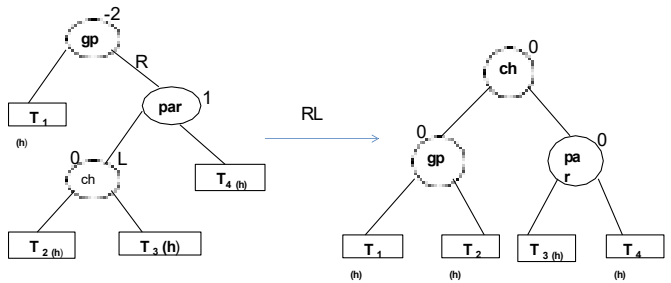
RL Rotation

There are basically four types of rotations which are as follows:

4. R L rotation : Inserted node is in the left subtree of right subtree of A. (Example: A, C, B.)



Contd ...



Insertion in AVL Tree

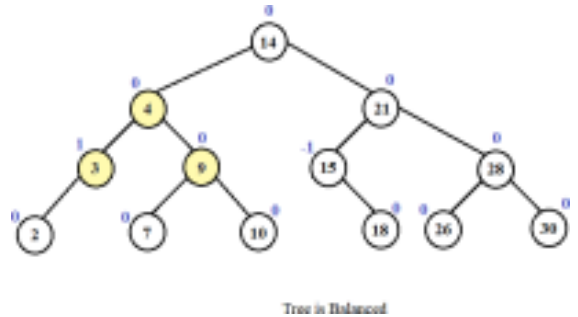
Let the newly inserted node be w

1. Perform standard BST insert for w.
2. Starting from w, travel up and find the first unbalanced node. Let z be the first unbalanced node, y be the child of z that comes on the path from w to z and x be the grandchild of z that comes on the path from w to z.
3. Re-balance the tree by performing appropriate rotations on the subtree rooted with z. There can be 4 possible cases that need to be handled as x, y and z can be arranged in 4 ways.
4. Following are the possible 4 arrangements:
   - y is the left child of z and x is the left child of y (Left Left Case)
   - y is the left child of z and x is the right child of y (Left Right Case)
   - y is the right child of z and x is the right child of y (Right Right Case)
   - y is the right child of z and x is the left child of y (Right Left Case)

Program Logic - Insert

```
struct Node* insert(struct Node* node, int key) {
    if (node == NULL)
        return newNode(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;   // No duplicates
    node->height = 1 + max(height(node->left),  height(node->right));
    int balance = getBalance(node);
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}
```

Construct AVL tree for the following data
21,26,30,9,4,14,28,18,15,10,2,3,7



Tree is Balanced

Deletion in AVL Tree

Let w be the node to be deleted

1. Perform standard BST delete for w.
2. Starting from w, travel up and find the first unbalanced node. Let z be the first unbalanced node, y be the larger height child of z, and x be the larger height child of y. Note that the definitions of x and y are different from insertion here.
3. Re-balance the tree by performing appropriate rotations on the subtree rooted with z. There can be 4 possible cases that needs to be handled as x, y and z can be arranged in 4 ways. Following are the possible 4 arrangements:
   - y is left child of z and x is left child of y (Left Left Case)
   - y is left child of z and x is right child of y (Left Right Case)
   - y is right child of z and x is right child of y (Right Right Case)
   - y is right child of z and x is left child of y (Right Left Case)

Case	Situation	Action
0/1 child	Simple delete	Replace or remove
2 children	Use successor	Copy + delete
LL	Left heavy	Right rotate
LR	Left-Right	Left + Right
RR	Right heavy	Left rotate
RL	Right-Left	Right + Left

Program Logic - Delete

```
struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL)
        return NULL;
    /* 1. Perform BST delete */
    if (key < root->key)
        root->left = deleteNode(root->left, key);
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    else {
        /* Node with 0 or 1 child */
        if (root->left == NULL || root->right == NULL) {
            struct Node* temp = (root->left) ? root->left : root->right;
            free(root);
            return temp;
        }
        /* Node with 2 children */
        struct Node* temp = minValueNode(root->right);
        root->key = temp->key;
        root->right = deleteNode(root->right, temp->key);
    }
```


Program Logic - Delete

```
    /* 2. Update height */
    root->height = 1 + max(height(root->left), height(root->right));
    /* 3. Get balance factor */
    int balance = getBalance(root);
    /* 4. Balance the tree */
    if (balance > 1 && getBalance(root->left) >= 0)
        return rightRotate(root);          // LL
    if (balance > 1 && getBalance(root->left) < 0) {
        root->left = leftRotate(root->left);
        return rightRotate(root);          // LR
    }
    if (balance < -1 && getBalance(root->right) <= 0)
        return leftRotate(root);           // RR
    if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);           // RL
    }
    return root;
}
```


Searching for a Node in an AVL Tree

- Searching in an AVL tree is performed exactly the same way as it is performed in a binary search tree.
- Because of the height-balancing of the tree, the search operation takes O(log *n*) time to complete.
- Since the operation does not modify the structure of the tree, no special provisions need to be taken.

Program Logic - Search

```
struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->key == key)
        return root;
    if (key < root->key)
        return search(root->left, key);
    else
        return search(root->right, key);
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->key);
        preorder(root->left);
        preorder(root->right);
    }
}
```

Examples

- 30,35,38, 20, 10, 25, 22, 28, 24, 29, 40 and 50
- 12,14,15,17,19,22,23 and 27
- 99,76,66,54,34,32,22,20,18 and 10
- 67, 54, 45, 43, 40, 33, 31, 25 and 20
- 50,75,65,85,80,25,60,55,90 and 88
- 99,50,70,25,40,60 ,30
- 30,10,15,20,35,45,12,13,14,11 and 90

Applications of AVL Tree

1. It is used to index huge records in a database and also to efficiently search in that.
2. For all types of in-memory collections, including sets and dictionaries, AVL Trees are used.
3. Database applications, where insertions and deletions are less common but frequent data lookups are necessary
4. Software that needs optimized search.
5. It is applied in corporate areas and storyline games.

Advantages of AVL Tree

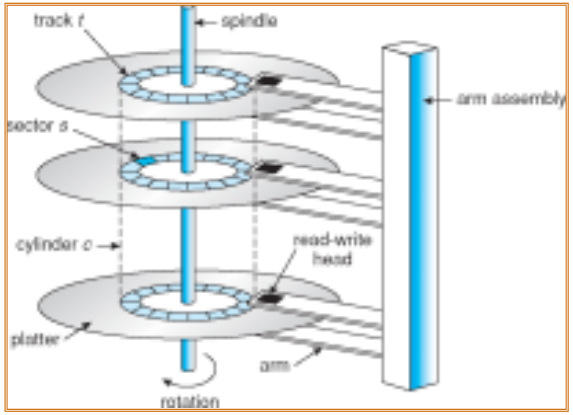
1. AVL trees can self-balance themselves.
2. It is surely not skewed.
3. It provides faster lookups than Red-Black Trees
4. Better searching time complexity compared to other trees like binary tree.
5. Height cannot exceed log(N), where, N is the total number of nodes in the tree.

Disadvantages of AVL Tree

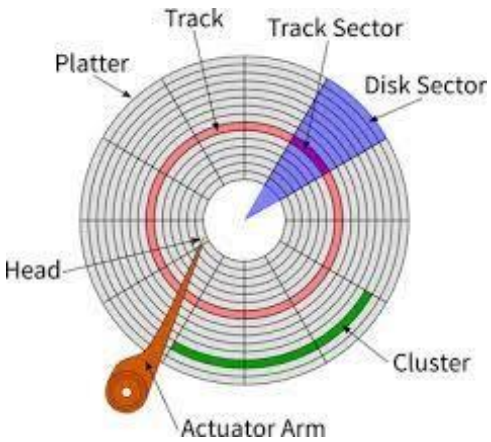
1. It is difficult to implement.
2. It has high constant factors for some of the operations.
3. Less used compared to Red-Black trees.
4. Due to its rather strict balance, AVL trees provide complicated insertion and removal operations as more rotations are performed.
5. Take more processing for balancing.

B-Tree Need

Contd ...



Contd ...



Contd ...

Platter → Surface → Track → Sector → Block

Let us assume block size is 512 Bytes.

Offset 0 511

Record size =126 Bytes.

# of records are 100.

# of records per block → 512/126 = 4

# of Blocks needed for 100 records are → 100/4 =25.

B-Tree

- A B-tree is self balancing multi-way search tree and a specialized m-way tree that is widely used for disk access. It is developed by Rudolf Bayer and Ed McCreight in 1970.
- A B-tree is designed to store sorted data and allows search, insert, and delete operations to be performed in logarithmic amortized time. A B-tree of order m (the maximum number of children that each node can have) is a tree with all the properties of an m-way search tree and in addition has the following properties:

B-Tree Properties

- **Maximum Children:** Every node has at most m children.
- **Maximum Keys:** A node can have at most m-1 keys.
- **Minimum Children (Non-Root, Non-Leaf):** Every non-leaf node, except for the root, must have at least $\lceil m/2 \rceil$ children.
- **Minimum Keys (Non-Root, Non-Leaf):** Consequently, every non-leaf node, except for the root, must have at least $\lceil m/2 \rceil - 1$ keys.
- **Root Node Properties:**
  - If the root is also a leaf node (meaning the tree contains only one node), it can have any number of keys (at least one) and no children.
  - If the root is a non-leaf node, it must have at least two children.
- **Leaf Node Level:** All leaf nodes are at the same level (depth) in the tree. This property ensures the tree remains balanced.
- **Key Ordering:** Keys within each node are stored in ascending order.

Insert Operation

In a B tree all insertions are done at the leaf node level. A new value is inserted in the B tree using the algorithm given below.

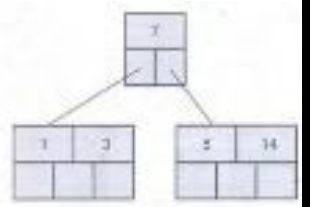
1. Search the B tree to find the leaf node where the new key value should be inserted.
2. If the leaf node is not full, that is it contains less than m-1 key values then insert the new element in the node, keeping the node's elements ordered.
3. If the leaf node is full, that is the leaf node already contain m-1 key values then
   a) insert the new value in order into the existing set of keys.
   b) split the node at its median into two nodes. note that the split nodes are half full.
   c) Push the median element up to its parent's node. If the parent's node is already full, then split the parent node by following the same steps.

Example-1

3, 14, 7, 1, 8, 5, 11, 17, 13, 6, 23, 12, 20, 26, 4, 16, 18, 24, 25, 19.

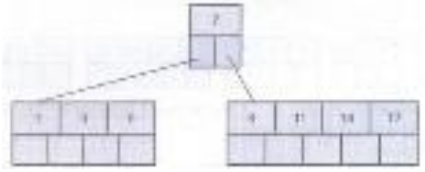
Insert 3, 14, 7, 1 as follows.

If we insert 8 then we need to split the node 1, 3, 7, 8, 14 at medium.

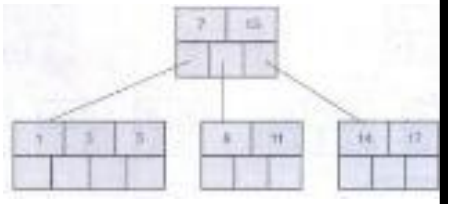


Contd …

Insert 5, 11, 7 which can be easily inserted in a B-tree.

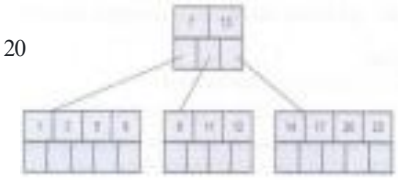


Now insert 13. But if we insert 13 then the leaf node will have 5 keys which is not allowed. Hence 8, 11,13, 14, 17 is split and medium node 13 is moved up.

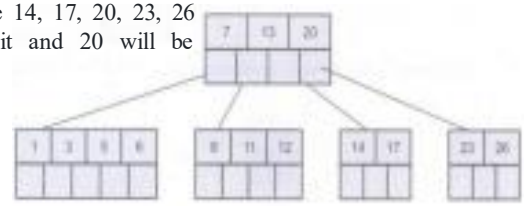


Contd …

Now insert 6, 23, 12, 20 without any split.



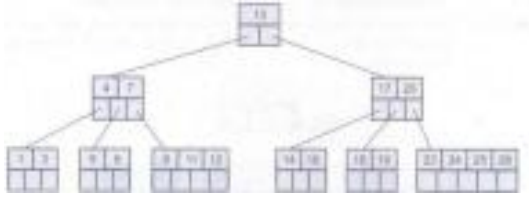
The 26 is inserted to the rightmost leaf node. Hence 14, 17, 20, 23, 26 the node is split and 20 will be moved up.



Contd …

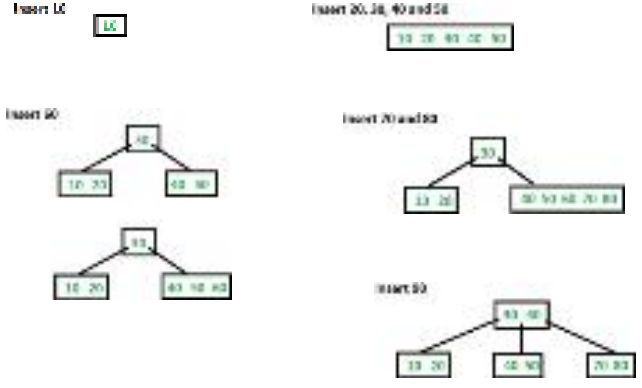
Insertion of node 4 causes left most node to split. The 1, 3, 4, 5, 6 causes key 4 to move up. Then insert 16, 18, 24, 25.

Finally insert 19. Then 4, 7, 13, 19, 20 needs to be split. The median 13 will be moved up to form a root node.



Example-2

Example : Insert 10, 20, 30, 40, 50, 60, 70, 80 and 90 in an initially empty B-Tree of order 6.

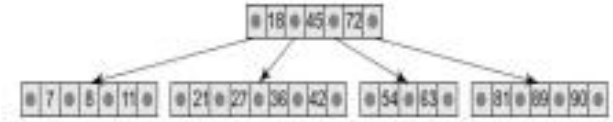


Example-3

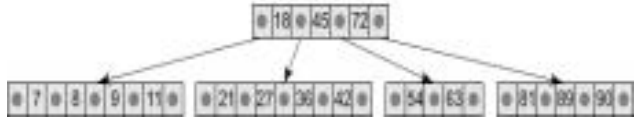
Look at the B tree of order 5 given below and insert 8, 9, 39, 4 into it.

Contd ...

Step 1: Insert 8

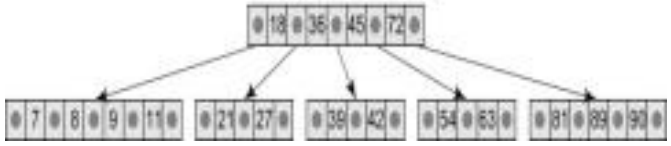


Step 2: Insert 9

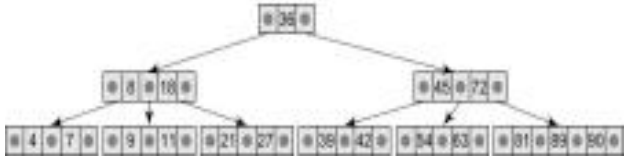


Contd ...

Step 3: Insert 39



Step 4: Insert 4



Delete Operation

Like insertion, deletion is also done from the leaf nodes. There are two cases of deletion. First, a leaf node has to be deleted. Second, an internal node has to be deleted. Let us first see the steps involved in deleting a leaf node.

1. Locate the leaf node which has to be deleted.
2. If the leaf node contains more than minimum number of key values (more than m/2 elements), then delete the value.
3. Else, if the leaf node does not contain even m/2 elements then, fill the node by taking an element either from the left or from the right sibling.
   a) If the left sibling has more than the minimum number of key values, push its largest key into its parent's node and pull down the intervening element from the parent node to the leaf node where the key is deleted.
   b) Else, if the right sibling has more than the minimum number of key values, push its smallest key into its parent node and pull down the intervening element from the parent node to the leaf node where the key is deleted.

Contd …

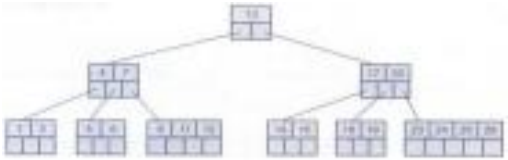
4. Else, if both left and right siblings contain only minimum number of elements, then create a new leaf node by combining the two leaf nodes and the intervening element of the parent node (ensuring that the number of elements do not exceed the maximum number of elements a node can have, that is, m). If pulling the intervening element from the parent node leaves it with less than minimum number of keys in the node, then propagate the process upwards thereby reducing the height of the B tree.
5. To delete an internal node, promote the successor or predecessor of the key to be deleted to occupy the position of the deleted key. This predecessor or successor will always be in the leaf node. So further the processing will be done as if a value from the leaf node has been deleted.

Example-1

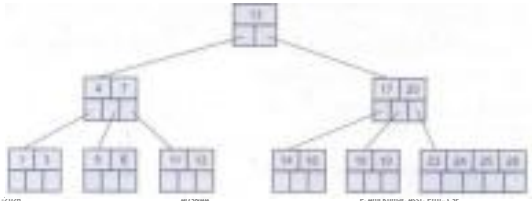
Order 5:
Delete the elements in order as
64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 16, 50,

Example-2

Consider the B-tree as

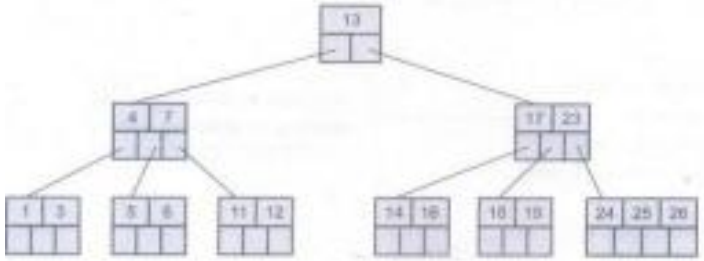


If we want to delete 8 then it is very simple



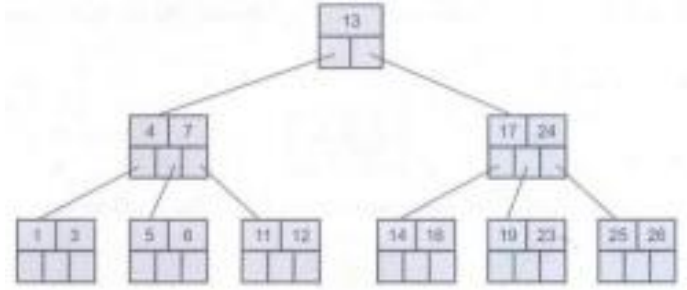
Contd …

Now we will delete 20, the 20 is not in a leaf node so we will find its successor which is 23. Hence 23 will be moved up to replace 20.



Contd …

Next we will delete 18. Deletion of 18 from the corresponding node causes the node with only one key, which is not desired (as per rule 4) in B-tree of order 5. The sibling node to immediate right has an extra key. In such a case we can borrow a key from parent and move spare key of sibling to up. See the following figure.

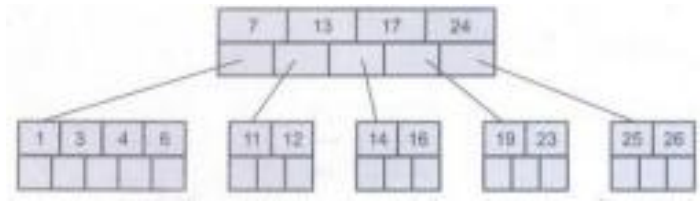


Contd …

Now delete 5. But deletion of 5 is not easy. The first thing is 5 is from leaf node. Secondly this leaf node has no extra keys nor siblings to immediate left or right. In such a situation we can combine this node with one of the siblings. That means remove 5 and combine 6 with the node 1, 3. To make the tree balanced we have to move parent's key down. Hence we will move 4 down as 4 is between 1, 3 and 6.

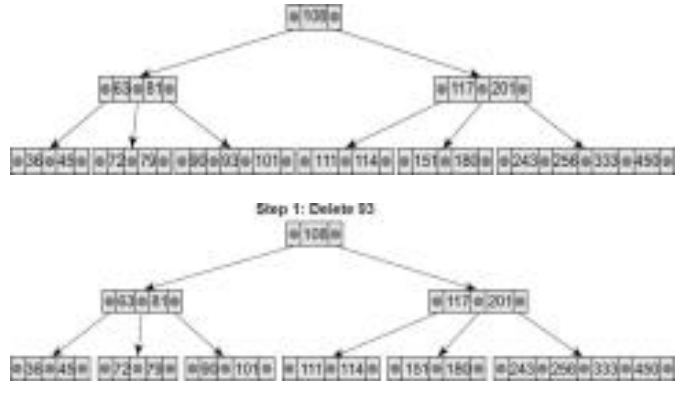
But again internal node of 7 contains only one key which not allowed in B-tree (as per rule 3). We then will try to borrow a key from sibling. But sibling 17, 24 has no spare key. Hence what we can do is that, combine 7 with 13 and 17, 24. Hence the B-tree will be

Contd …



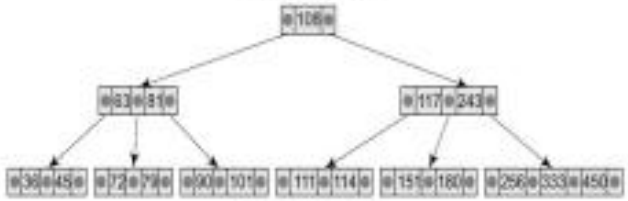
Example-2

Consider the B tree of order 5 and delete the values - 93, 201, 180, 72 from it.

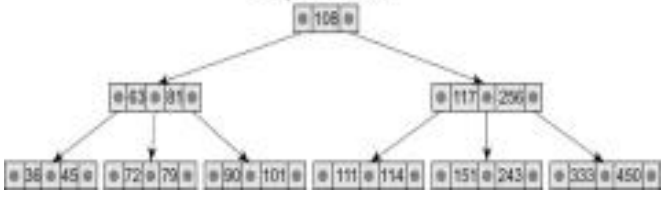


Contd ...

Step 2: Delete 201

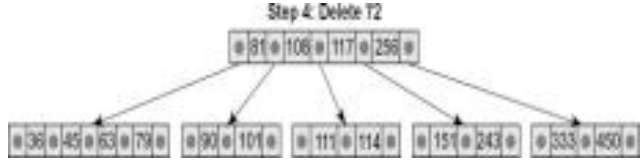


Step 3: Delete 180



Contd ...

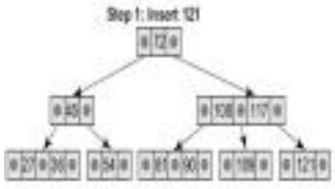
Step 4: Delete 72



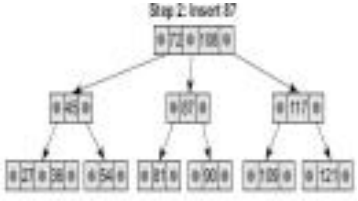
Example: Consider the B-tree of order 3 given below and perform the following operations: (a) insert 121, 87 and then (b) delete 36, 109.

Contd ...

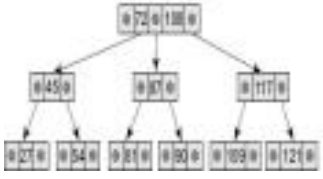
Step 1: Insert 121



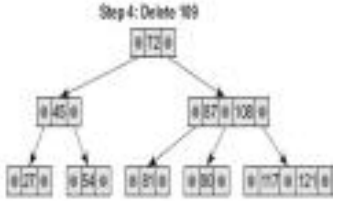
Step 2: Insert 87



Step 3: Delete 36



Step 4: Delete 109



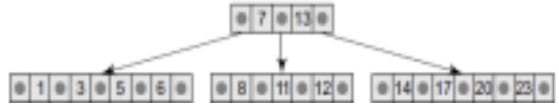
Contd ...

Example: Create a B-tree of order 5 by inserting the following elements: 3, 14, 7, 1, 8, 5, 11, 17, 13, 6, 23, 12, 20, 26, 4, 16, 18, 24, 25, and 19.

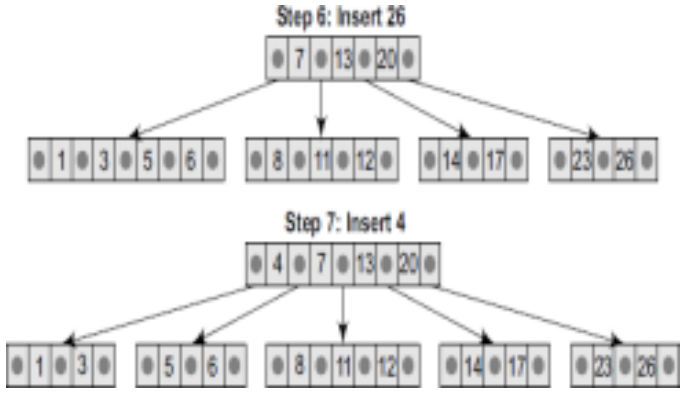
Step 1: Insert 3, 14, 7, 1 Step 2: Insert 8 Step 3: Insert 5, 11, 17

Step 4: Insert 13

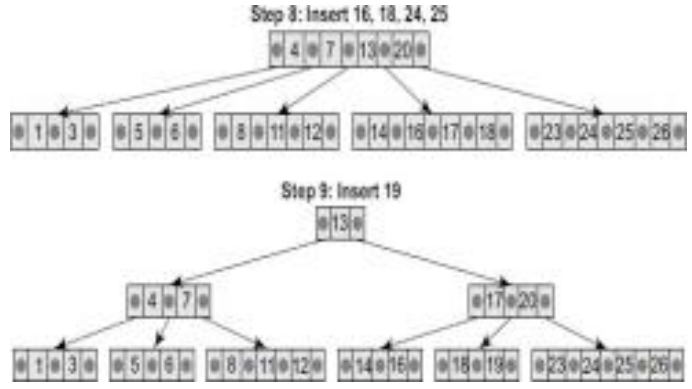
Step 5: Insert 6, 23, 12, 20



Contd …



Contd …

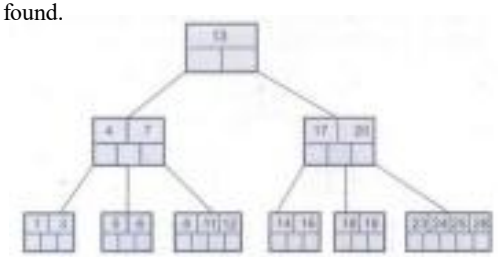


Search Operation

- Searching for an element in a B-trees is similar to that in m-way search trees.

If we want to search node 11 then

1. 11 13 Hence search left node
2. 11 > 7: Hence rightmost node
3. 11 > 8, move in second block
4. node 11 is found.



Applications of B-Tree

1. It is used in large databases to access data stored on the disk
2. Searching for data in a data set can be achieved in significantly less time using the B-Tree
3. With the indexing feature, multilevel indexing can be achieved.
4. Most of the servers also use the B-tree approach.
5. B-Trees are also used in other areas such as natural language processing, computer networks, and cryptography.

Advantages of B-Tree

1. B-Trees have a guaranteed time complexity of $O(\log_m n)$ for basic operations like insertion, deletion, and searching, which makes them suitable for large data sets and real-time applications.
2. B-Trees are self-balancing.
3. High-concurrency and high-throughput.
4. Efficient storage utilization.

Disadvantages of B-Tree

1. B-Trees are based on disk-based data structures and can have a high disk usage.
2. Not the best for all cases.
3. Slow in comparison to other data structures.

Abstract Data Type: Priority Queue

A priority queue is a *collection* of zero or more items.
- associated with each item its priority

A priority queue has the following operations
- *insert*(item i) (enqueue) a new item
- *delete*() (dequeue) the member with the highest priority
- *find*() the item with the highest priority
- decreasePriority(item i, p) decrease the priority of ith item to p

Note that in a priority queue "*first in first out*" does not apply in general.
If the priority is same, then follow the queue principle.

Priority Queues: Assumptions

The highest priority can be either the *minimum* value of all the *items*, or the *maximum*.

Assume the priority queue has *n* elements

Priority Queue Representations

1. Linked lists
2. Arrays.
3. Binary Heaps
4. Binomial Heaps

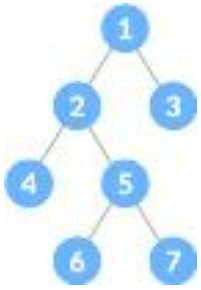
Basic Terms

Full Binary Tree:
A full binary tree is a binary tree in which **all of the nodes have either 0 or 2 offspring**. In other terms, a full binary tree is a binary tree in which all nodes, except the leaf nodes, have two offspring.

Complete Binary Tree:
A Binary Tree is a Complete Binary Tree if all the levels are completely filled except possibly the last level and the last level has **all keys as left as possible**.

Perfect Binary Tree:
A Binary tree is a Perfect Binary Tree in which **all the internal nodes have two children** and **all leaf nodes are at the same level**.

Guess, this tree is?



Heap

A heap is a certain kind of complete binary tree.

Contd …

Root

A heap is a certain kind of complete binary tree.

When a complete binary tree is built, its first node must be the root.

Contd …

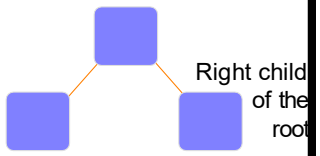
Complete binary tree.

Left child of the root

The second node is always the left child of the root.

Contd …

Complete binary tree.

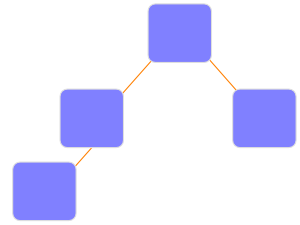


Right child of the root

The third node is always the right child of the root.

Contd …

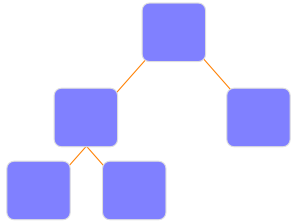
Complete binary tree.



The next nodes always fill the next level from left-to-right.

Contd ...

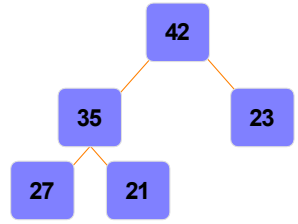
Complete binary tree.



The next nodes always fill the next level from left-to-right.

Implementing a Heap

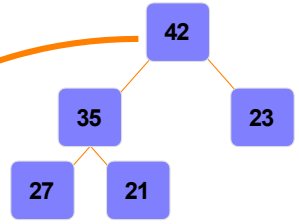
We will store the data from the nodes in a partially-filled array.



An array of data

Contd ...

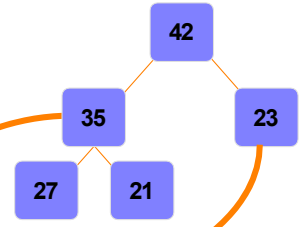
Data from the root goes in the first location of the array.

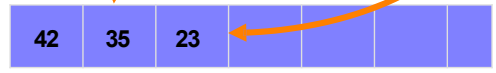


An array of data

Contd ...

Data from the next row goes in the next two array locations.

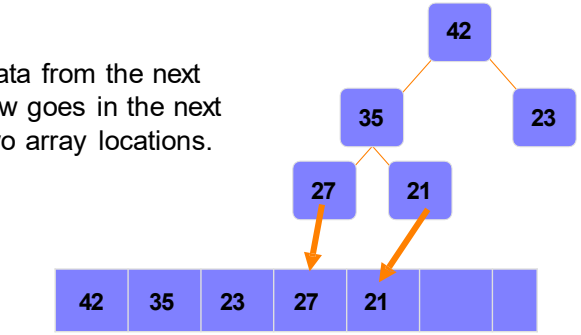




An array of data

Contd ...

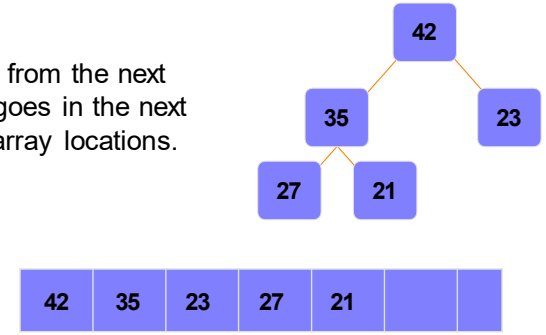
Data from the next row goes in the next two array locations.



An array of data

Contd ...

Data from the next row goes in the next two array locations.



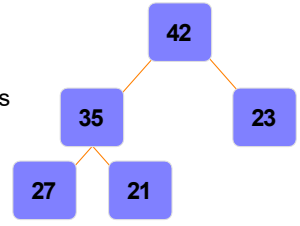
An array of data

We don't care what's in this part of the array.

Contd ...

The links between the tree's nodes are not actually stored as pointers, or in any other way.

The only way we "know" that "the array is a tree" is from the way we manipulate the data.

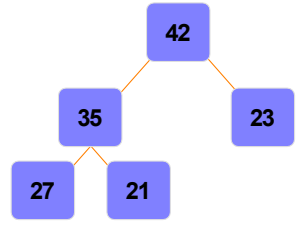


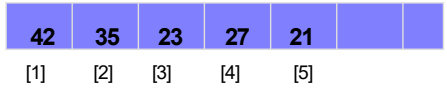
An array of data

Important points of Heaps

If index starts from 1
For node i: left = 2*i
right = (2*i)+1
parent = floor(i/2)

If index starts from 0
For node i: left = 2*i+1
right = (2*i)+2
parent = ceil(i/2)-1



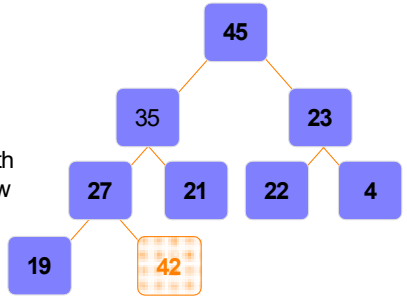


Heap Construction

1. Create a new node at the end of heap.
2. Assign new value to the node.
3. Compare the value of this child node with its parent.
   If value of parent is grater than child, then swap them.
   Otherwise, continue.
4. Repeat 3 until Heap property holds.

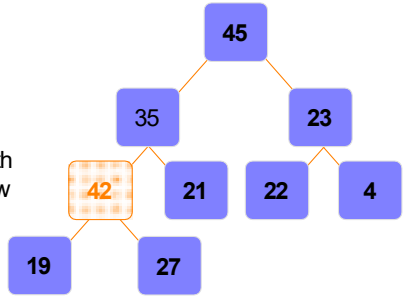
Adding a Node to a Heap

Put the new node in the next available spot.
Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



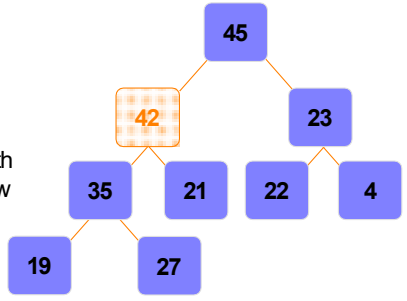
Contd ...

Put the new node in the next available spot.
Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



Contd ...

Put the new node in the next available spot.
Push the new node upward, swapping with its parent until the new node reaches an acceptable location.

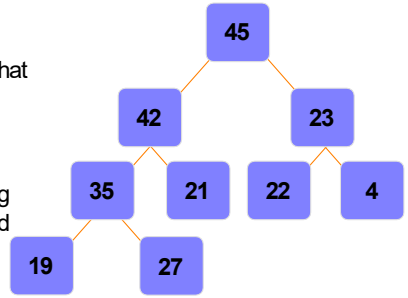


Contd …

The parent has a key that is >= new node, or

The node reaches the root.

The process of pushing the new node upward is called reheapification upward.

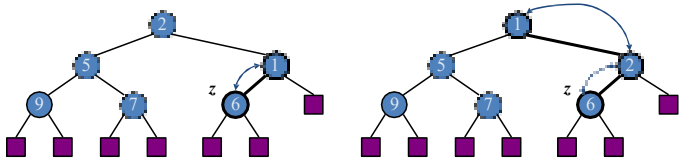


Insertion algorithm

1. Add a new element to the end of an array.
2. Sift up the new element, while heap property is broken. Sifting is done as following: compare node's value with parent's value. If they are in wrong order, swap them.

Upheap

- After the insertion of a new key k, the heap-order property may be violated
- Algorithm upheap restores the heap-order property by swapping k along an upward path from the insertion node
- Upheap terminates when the key k reaches the root or a node whose parent has a key smaller than or equal to k
- Since a heap has height $O(\log n)$, upheap runs in $O(\log n)$ time

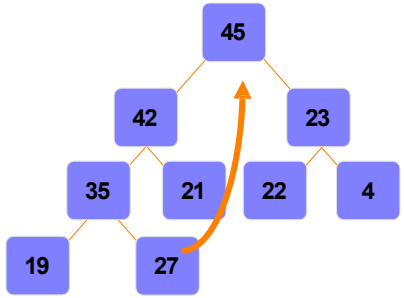


Program Logic - Insertion algorithm

```
void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}
void insertMaxHeap(int value) {
   maxHeap[maxSize] = value;
   maxHeapifyUp(maxSize);
   maxSize++;
}
void maxHeapifyUp(int i) {
   while (i > 0 && maxHeap[(i - 1) / 2] < maxHeap[i]) {
      swap(&maxHeap[i], &maxHeap[(i - 1) / 2]);
      i = (i - 1) / 2;
   }
}
```

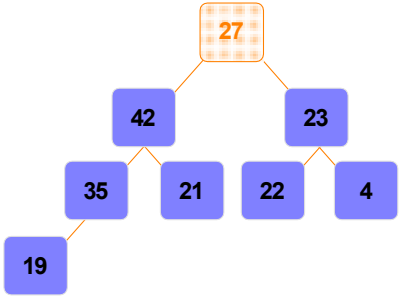
Removing the Top of a Heap

Move the last node onto the root.



Contd ...

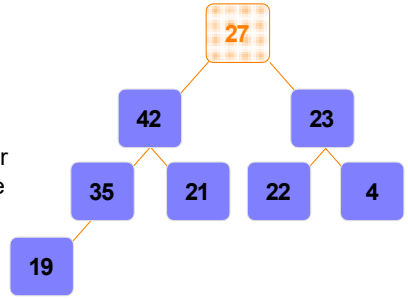
Move the last node onto the root.



Contd ...

Move the last node onto the root.

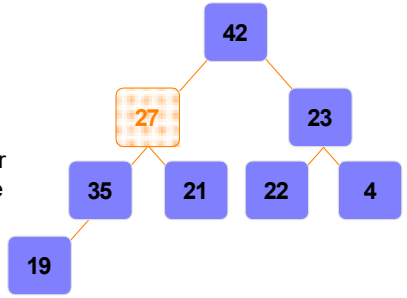
Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



Contd ...

Move the last node onto the root.

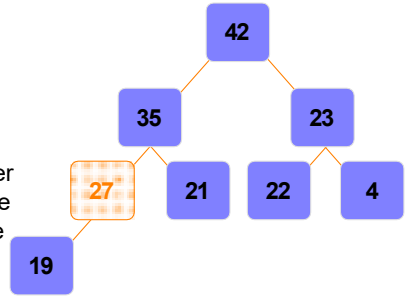
Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



Contd …

Move the last node onto the root.

Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.

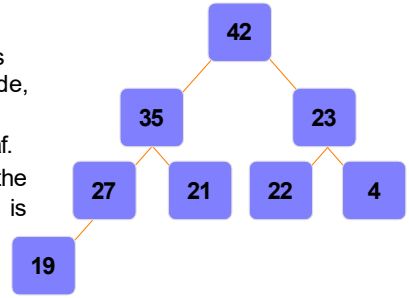


Contd …

The children all have keys <= the out-of-place node, or

The node reaches the leaf.

The process of pushing the new node downward is called reheapification downward.



Removal algorithm

1. Copy the last value in the array to the root;
2. Decrease heap's size by 1;
3. Sift down root's value. Sifting is done as following:
   a. if current node has no children, sifting is over;
   b. if current node has one child: check, if heap property is broken, then swap current node's value and child value; sift down the child;
   c. if current node has two children: find the smallest of them. If heap property is broken, then swap current node's value and selected child value; sift down the child.

Heapify

```
MAX-HEAPIFY(A,i)
  l = LEFT(i)
  r = RIGHT(i)
  if l <= A.heapsize and A[l] > A[i]
     largest = l
  else
     largest = i
  if r <= A.heapsize and A[r] > A[largest]
     largest = r
  if largest != i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A,largest)
```

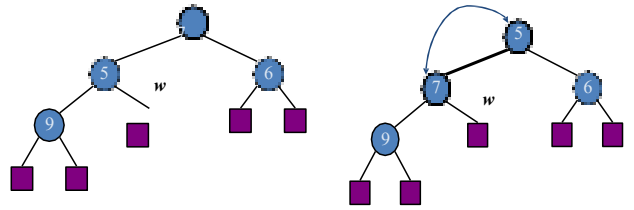
Time Complexity of Max-Heapify on a node of height h is O(h).

Program Logic - Deletion algorithm

```
void deleteMaxHeap() {
   if (maxSize == 0) {
      printf("Max Heap is empty\n");
      return;
   }
   printf("Deleted element: %d\n", maxHeap[0]);
   maxHeap[0] = maxHeap[--maxSize];
   maxHeapifyDown(0);
}
void maxHeapifyDown(int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
   if (left < maxSize && maxHeap[left] > maxHeap[largest])
      largest = left;
   if (right < maxSize && maxHeap[right] > maxHeap[largest])
      largest = right;
   if (largest != i) {
      swap(&maxHeap[i], &maxHeap[largest]);
      maxHeapifyDown(largest);
   }
}
```

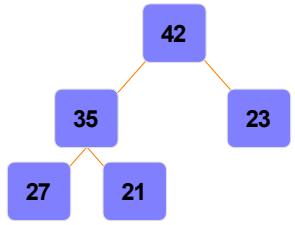

Downheap

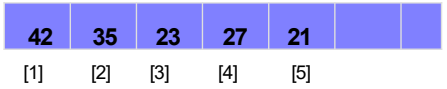
- After replacing the root key with the key k of the last node, the heap-order property may be violated
- Algorithm downheap restores the heap property by swapping key k with the child with the smallest key along a downward path from the root
- Downheap terminates when key k reaches a leaf or a node whose children have keys greater than or equal to k
- Since a heap has height $O(\log n)$, downheap runs in $O(\log n)$ time.



Display of Heap

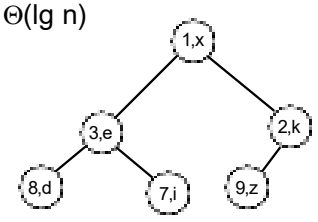
```
void displayMaxHeap() {
   printf("Max Heap: ");
   for (int i = 0; i < maxSize; i++)
      printf("%d ", maxHeap[i]);
   printf("\n");
}
```





Analysis of Heap

In the worst case *insert*() is Θ(lg n) and
deleteMin() is Θ(lg n)
findMin() is Θ(1)
decreaseKey(i, p) is Θ(lg n)



Heapsort

1. Build a heap from the given input array.
2. Repeat the following steps until the heap contains only one element:
   a) Swap the root element of the heap (which is the largest element) with the last element of the heap.
   b) Remove the last element of the heap (which is now in the correct position).
   c) Heapify the remaining elements of the heap.
3. The sorted array is obtained by reversing the order of the elements in the input array.

- 44, 33, 77, 11, 55, 88, 66
- 4, 10, 3, 5, 1
- 1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17

Applications of HEAPs

1. Heap Sort: Heap Sort uses Binary Heap to sort an array in O(nLogn) time.
2. Priority Queue: Priority queues can be efficiently implemented using Binary Heap because it supports insert(), delete() and extractmax(), decreaseKey() operations in O(logn) time.
3. Graph Algorithms: The priority queues are especially used in Graph Algorithms like Dijkstra's Shortest Path and Prim's Minimum Spanning Tree.
4. Many problems can be efficiently solved using Heaps. See following for example.
   a) K'th Largest Element in an array.
   b) Sort an almost sorted array
   c) Merge K Sorted Arrays.

Exercise: Heps

- Where may an item with the largest key be stored in a heap?
  True or False:
- A pre-order traversal of a heap will list out its keys in sorted order. Prove it is true or provide a counter example.
- Let H be a heap with 7 distinct elements (1,2,3,4,5,6, and 7). Is it possible that a preorder traversal visits the elements in sorted order? What about an inorder traversal or a postorder traversal? In each case, either show such a heap or prove that none exists.

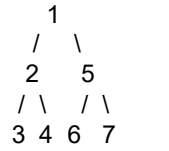


fig: pre-order

GATE bits

2020:

```
for i = 1 to n
   for j = i to n
      print("*")
```

2019:

Which hash function is **best to avoid clustering**?

A. Division method
B. Folding
C. Mid-square method
D. Multiplication method

GATE bits

2021:

Which technique avoids **both primary and secondary clustering**?

A. Linear probing
B. Quadratic probing
C. Double hashing
D. Chaining

2026:

In a double hashing scheme, $h_1(k)$= k mod 11 and $h_2(k)$=1+(k mod 7) are the auxiliary hash functions. The size m of the hash table is 11. The hash function for the *i*-th probe in the open address table is $[h_1(k)+ih_2(k)]$ mod m The following keys are inserted in the given order: 63, 50, 25, 79, 67, 24.

The slot at which key 24 gets stored is _____. (Answer in integer)

GATE bits

2017:

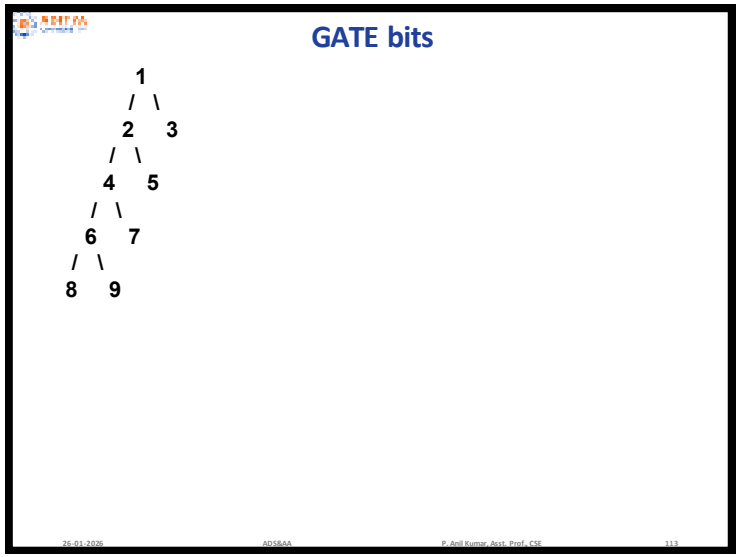
Let T be a binary search tree with 15 nodes. The minimum and maximum possible heights of T are:
Note: The height of a tree with a single node is 0.

2020:

What is the worstcase time complexity of inserting n^2 elements into an AVL-tree with n elements initially?

2018:

The postorder traversal of a binary tree is 8, 9, 6, 7, 4, 5, 2, 3, 1. The inorder traversal of the same tree is 8, 6, 9, 4, 7, 2, 5, 1, 3. The height of a tree is the length of the longest path from the root to any leaf. The height of the binary tree above is _____ .


GATE bits
1
2 3
4 5
6 7
8 9
26-01-2026
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
113